

1 0

10 Lists

10.0.1 Variables with more than one value

You have already seen ordinary variables that store a single value. However other variable types can hold more than one value. These are called containers because they can contain more than one object. The simplest type is called a list. Here is an example of a list being used:

```
which_one = int(input("What month (1-12)? "))
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
          'August', 'September', 'October', 'November', 'December']

if 1 <= which_one <= 12:
    print("The month is", months[which_one - 1])
```

and an output example:

```
What month (1-12)? 3
The month is March
```

In this example the `months` is a list. `months` is defined with the lines `months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']` (note that a `\` could also be used to split a long line, but that is not necessary in this case because Python is intelligent enough to recognize that everything within brackets belongs together). The `[` and `]` start and end the list with commas `(,)` separating the list items. The list is used in `months[which_one - 1]`. A list consists of items that are numbered starting at 0. In other words if you wanted January you would use `months[0]`. Give a list a number and it will return the value that is stored at that location.

The statement `if 1 <= which_one <= 12:` will only be true if `which_one` is between one and twelve inclusive (in other words it is what you would expect if you have seen that in algebra).

Lists can be thought of as a series of boxes. Each box has a different value. For example, the boxes created by `demolist = ['life', 42, 'the universe', 6, 'and', 9]` would look like this:

box number	0	1	2	3	4	5
demolist	"life"	42	"the universe"	6	"and"	9

Each box is referenced by its number so the statement `demolist[0]` would get `'life'`, `demolist[1]` would get 42 and so on up to `demolist[5]` getting 9.

10.0.2 More features of lists

The next example is just to show a lot of other stuff lists can do (for once I don't expect you to type it in, but you should probably play around with lists in interactive mode until you are comfortable with them.). Here goes:

```
demolist = ["life", 42, "the universe", 6, "and", 9]
print("demolist = ",demolist)
demolist.append("everything")
print("after 'everything' was appended demolist is now:")
print(demolist)
print("len(demolist) =", len(demolist))
print("demolist.index(42) =", demolist.index(42))
print("demolist[1] =", demolist[1])

# Next we will loop through the list
for c in range(len(demolist)):
    print("demolist[" + c + "] =", demolist[c])

del demolist[2]
print("After 'the universe' was removed demolist is now:")
print(demolist)
if "life" in demolist:
    print("'life' was found in demolist")
else:
    print("'life' was not found in demolist")

if "amoeba" in demolist:
    print("'amoeba' was found in demolist")

if "amoeba" not in demolist:
    print("'amoeba' was not found in demolist")

another_list = [42,7,0,123]
another_list.sort()
print("The sorted another_list is", another_list)
```

The output is:

```
demolist = ['life', 42, 'the universe', 6, 'and', 9]
after 'everything' was appended demolist is now:
['life', 42, 'the universe', 6, 'and', 9, 'everything']
len(demolist) = 7
demolist.index(42) = 1
demolist[1] = 42
demolist[ 0 ] = life
demolist[ 1 ] = 42
demolist[ 2 ] = the universe
demolist[ 3 ] = 6
demolist[ 4 ] = and
demolist[ 5 ] = 9
demolist[ 6 ] = everything
After 'the universe' was removed demolist is now:
['life', 42, 6, 'and', 9, 'everything']
'life' was found in demolist
'amoeba' was not found in demolist
The sorted another_list is [0, 7, 42, 123]
```

This example uses a whole bunch of new functions. Notice that you can just `print` a whole list. Next the `append` function is used to add a new item to the end of the list. `len` returns how many items are in a list. The valid indexes (as in numbers that can be used inside of the `[]`) of a list range from 0 to `len - 1`. The `index` function tells where the first location

of an item is located in a list. Notice how `demolist.index(42)` returns 1, and when `demolist[1]` is run it returns 42. To get help on all the functions a list provides for you, type `help(list)` in the interactive Python interpreter.

The line `# Next we will loop through the list` is just a reminder to the programmer (also called a *comment*). Python ignores everything that is written after a `#` on the current line. Next the lines:

```
for c in range(len(demolist)):
    print('demolist[' + c + '] =', demolist[c])
```

create a variable `c`, which starts at 0 and is incremented until it reaches the last index of the list. Meanwhile the `print` statement prints out each element of the list.

A much better way to do the above is:

```
for c, x in enumerate(demolist):
    print("demolist[" + c + "] =", x)
```

The `del` command can be used to remove a given element in a list. The next few lines use the `in` operator to test if an element is in or is not in a list. The `sort` function sorts the list. This is useful if you need a list in order from smallest number to largest or alphabetical. Note that this rearranges the list. In summary, for a list, the following operations occur:

example	explanation
<code>demolist[2]</code>	accesses the element at index 2
<code>demolist[2] = 3</code>	sets the element at index 2 to be 3
<code>del demolist[2]</code>	removes the element at index 2
<code>len(demolist)</code>	returns the length of <code>demolist</code>
<code>"value" in demolist</code>	is <i>True</i> if <code>"value"</code> is an element in <code>demolist</code>
<code>"value" not in demolist</code>	is <i>True</i> if <code>"value"</code> is not an element in <code>demolist</code>
<code>another_list.sort()</code>	sorts <code>another_list</code> . Note that the list must be all numbers or all strings to be sorted.
<code>demolist.index("value")</code>	returns the index of the first place that <code>"value"</code> occurs
<code>demolist.append("value")</code>	adds an element <code>"value"</code> at the end of the list
<code>demolist.remove("value")</code>	removes the first occurrence of <code>value</code> from <code>demolist</code> (same as <code>del demolist[demolist.index("value")]</code>)

This next example uses these features in a more useful way:

```
menu_item = 0
namelist = []
while menu_item != 9:
    print("-----")
    print("1. Print the list")
    print("2. Add a name to the list")
    print("3. Remove a name from the list")
    print("4. Change an item in the list")
    print("9. Quit")
    menu_item = int(input("Pick an item from the menu: "))
    if menu_item == 1:
        current = 0
        if len(namelist) > 0:
```

```

        while current < len(namelist):
            print(current, ".", namelist[current])
            current = current + 1
        else:
            print("List is empty")
    elif menu_item == 2:
        name = input("Type in a name to add: ")
        namelist.append(name)
    elif menu_item == 3:
        del_name = input("What name would you like to remove: ")
        if del_name in namelist:
            # namelist.remove(del_name) would work just as fine
            item_number = namelist.index(del_name)
            del namelist[item_number]
            # The code above only removes the first occurrence of
            # the name. The code below from Gerald removes all.
            # while del_name in namelist:
            #     item_number = namelist.index(del_name)
            #     del namelist[item_number]
        else:
            print(del_name, "was not found")
    elif menu_item == 4:
        old_name = input("What name would you like to change: ")
        if old_name in namelist:
            item_number = namelist.index(old_name)
            new_name = input("What is the new name: ")
            namelist[item_number] = new_name
        else:
            print(old_name, "was not found")

print("Goodbye")

```

And here is part of the output:

```

-----
1. Print the list
2. Add a name to the list
3. Remove a name from the list
4. Change an item in the list
9. Quit

Pick an item from the menu: 2
Type in a name to add: Jack

Pick an item from the menu: 2
Type in a name to add: Jill

Pick an item from the menu: 1
0 . Jack
1 . Jill

Pick an item from the menu: 3
What name would you like to remove: Jack

Pick an item from the menu: 4
What name would you like to change: Jill
What is the new name: Jill Peters

Pick an item from the menu: 1
0 . Jill Peters

Pick an item from the menu: 9
Goodbye

```

That was a long program. Let's take a look at the source code. The line `namelist = []` makes the variable `namelist` a list with no items (or elements). The next important line is `while menu_item != 9:`. This line starts a loop that allows the menu system for this program. The next few lines display a menu and decide which part of the program to run.

The section

```
current = 0
if len(namelist) > 0:
    while current < len(namelist):
        print(current, ".", namelist[current])
        current = current + 1
else:
    print("List is empty")
```

goes through the list and prints each name. `len(namelist)` tells how many items are in the list. If `len` returns 0, then the list is empty.

Then, a few lines later, the statement `namelist.append(name)` appears. It uses the `append` function to add an item to the end of the list. Jump down another two lines, and notice this section of code:

```
item_number = namelist.index(del_name)
del namelist[item_number]
```

Here the `index` function is used to find the index value that will be used later to remove the item. `del namelist[item_number]` is used to remove an element of the list.

The next section

```
old_name = input("What name would you like to change: ")
if old_name in namelist:
    item_number = namelist.index(old_name)
    new_name = input("What is the new name: ")
    namelist[item_number] = new_name
else:
    print(old_name, "was not found")
```

uses `index` to find the `item_number` and then puts `new_name` where the `old_name` was.

Congratulations, with lists under your belt, you now know enough of the language that you could do any computations that a computer can do (this is technically known as Turing-Completeness¹). Of course, there are still many features that are used to make your life easier.

10.0.3 Examples

test.py

```
## This program runs a test of knowledge

# First get the test questions
# Later this will be modified to use file io.
def get_questions():
    # notice how the data is stored as a list of lists
    return ["What color is the daytime sky on a clear day? ", "blue"],
```

1 https://en.wikipedia.org/wiki/Turing_completeness

```
        ["What is the answer to life, the universe and everything? ", "42"],
        ["What is a three letter word for mouse trap? ", "cat"]]

# This will test a single question
# it takes a single question in
# it returns True if the user typed the correct answer, otherwise False

def check_question(question_and_answer):
    # extract the question and the answer from the list
    # This function takes a list with two elements, a question and an answer.
    question = question_and_answer[0]
    answer = question_and_answer[1]
    # give the question to the user
    given_answer = input(question)
    # compare the user's answer to the tester's answer
    if answer == given_answer:
        print("Correct")
        return True
    else:
        print("Incorrect, correct was:", answer)
        return False

# This will run through all the questions
def run_test(questions):
    if len(questions) == 0:
        print("No questions were given.")
        # the return exits the function
        return
    index = 0
    right = 0
    while index < len(questions):
        # Check the question
        #Note that this is extracting a question and answer list from the list
        # of lists.
        if check_question(questions[index]):
            right = right + 1
        # go to the next question
        index = index + 1
    # notice the order of the computation, first multiply, then divide
    print("You got", right * 100 / len(questions),\
          "% right out of", len(questions))

# now let's get the questions from the get_questions function, and
# send the returned list of lists as an argument to the run_test function.

run_test(get_questions())
```

The values True and False point to 1 and 0, respectively. They are often used in sanity checks, loop conditions etc. You will learn more about this a little bit later (chapter Boolean Expressions²). Please note that `get_questions()` is essentially a list because even though it's technically a function, returning a list of lists is the only thing it does.

Sample Output:

```
What color is the daytime sky on a clear day? green
Incorrect, correct was: blue
What is the answer to life, the universe and everything? 42
```

² Chapter 12 on page 67